

Git for Developers

Module 10 — Enterprise Git

Code Reviews, Protected Branches, Monorepos, LFS, and Security

Module 10 — Agenda

Team and Governance

- Code reviews and Pull Requests at scale
- Protected branches and merge policies

Scale and Security

- Monorepos and scaling Git for large organizations
- Git LFS: handling large binary files
- Security: signed commits with GPG and SSH

Part 1 — Code Reviews & Pull Requests

The Code Review Process

Developer: push feature branch

Open Pull Request (PR) / Merge Request (MR)

Automated checks (CI):

- Unit & integration tests
- Linting and code style
- Security scanning (SAST)
- Dependency vulnerability audit

Human review: feedback, approval

Effective Code Reviews

For reviewers

- Focus on **correctness, security, and maintainability** — not style (use a linter)
- Be constructive: suggest, don't demand
- Approve when the code is **good enough**, not perfect
- Review in small sessions — don't review 1,000 lines at once

For authors

- Keep PRs **small** (< 400 lines changed where possible)
- Add a clear **PR description** — what changed, why, how to test

PR Description Template

Summary

Brief description of what this PR does and why.

Changes

- Added JWT refresh token rotation
- Deprecated legacy session-based auth
- Updated API docs

Testing

1. Run `npm test` – all tests pass
2. Verify login flow in staging: <https://staging.example.com/login>

Notes

Part 2 — Protected Branches & Policies

Branch Protection Rules

On GitHub, GitLab, and Bitbucket, protect important branches:

Rule	What it prevents
Require PR before merging	Direct pushes to <code>main</code>
Require N approvals	Merging without peer review
Require status checks	Merging when CI fails
Require linear history	Merge commits (enforces rebase/squash)

Enforcing Conventional Commits

Example: GitHub Actions workflow to validate commit messages

```
name: Lint PR Title
```

```
on:
```

```
  pull_request:
```

```
    types: [opened, edited, synchronize]
```

```
jobs:
```

```
  lint:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - uses: actions/checkout@v4
```

```
      - uses: amannn/action-semantic-pull-request@v5
```

```
      with:
```

Merge Strategies – Which to Choose?

Strategy	History	When to use
Merge commit	Preserves all commits + merge commit	Feature branches, when history matters
Squash & merge	All PR commits → one commit on <code>main</code>	Clean <code>main</code> history, messy PRs
Rebase & merge	Linear — PR commits replayed on <code>main</code>	Linear history preference, clean PRs

```
# Configure default merge strategy per repository
```

```
# (GitHub: Repo Settings -> General -> Allow merge commits / squash / rebase)
```

Part 3 — Monorepos & Scaling Git

What is a Monorepo?

A **monorepo** stores multiple projects or services in a single Git repository:

```
my-monorepo/  
├── services/  
│   ├── auth-service/  
│   ├── payment-service/  
│   └── notification-service/  
├── packages/  
│   ├── shared-ui/  
│   └── common-utils/  
├── infrastructure/  
│   └── terraform/
```

Monorepo – Pros and Cons

	Pros	Cons
Code sharing	Easy to share libraries across services	Harder to enforce boundaries
Atomic changes	One PR changes service + shared lib together	Large <code>git clone</code> and <code>git log</code>
Dependency management	Single lock file, consistent versions	Build times grow with repo size
Tooling	Specialized tools needed	Additional complexity

Scaling Git for Monorepos

```
# Sparse checkout – only check out the paths you need  
git clone --filter=blob:none --sparse https://github.com/org/monorepo.git  
cd monorepo  
git sparse-checkout init --cone  
git sparse-checkout set services/auth-service packages/common-utils  
  
# Virtual File System for Git (VFS for Git / Scalar)  
# Microsoft's solution for very large repos (Windows, Office)  
scalar clone https://github.com/org/large-monorepo.git
```

Popular monorepo tools

Tool	Language	Features
------	----------	----------

Part 4 — Git LFS

What is Git LFS?

Git Large File Storage replaces large binary files in the repository with text pointers, storing the actual content on a separate server.

Without LFS:

`.git/objects/...`

├─ design-v3.psd (48 MB) →
├─ video-demo.mp4 (250 MB) →
└─ logo.ai (12 MB) →

`git clone` → downloads 310 MB

With LFS:

`.git/objects/...`

├─ pointer to LFS server (200 bytes)
├─ pointer to LFS server (200 bytes)
└─ pointer to LFS server (200 bytes)

`git clone` → downloads ~1 KB of pointers
(actual files fetched on checkout)

Setting Up Git LFS

Install Git LFS

`brew install git-lfs` *# macOS*

`sudo apt install git-lfs` *# Ubuntu*

Initialize LFS in a repository

`git lfs install`

Track file types with LFS

`git lfs track "*.psd"`

`git lfs track "*.mp4"`

`git lfs track "*.ai"`

`git lfs track "assets/large/**"`

Git LFS — Day to Day

Git LFS is mostly transparent — regular git commands work

```
git add design-v3.psd
```

```
git commit -m "assets: update logo design"
```

```
git push      # pointers pushed to git; binaries pushed to LFS server
```

Check which files are tracked by LFS

```
git lfs ls-files
```

Check LFS status

```
git lfs status
```

Pull LFS files explicitly (if not auto-downloaded)

```
git lfs pull
```

Part 5 — Security & Signed Commits

Why Sign Commits?

Anyone can set any name and email in `git config user.name`. Without signatures:

- A commit claiming to be from `cto@company.com` could be from anyone
- Supply chain attacks can inject malicious commits into OSS projects

Signing attaches a cryptographic proof that **you** created the commit.

Signing with GPG

Generate a GPG key

```
gpg --full-generate-key
```

List your GPG keys

```
gpg --list-secret-keys --keyid-format LONG
```

Configure Git to use your GPG key

```
git config --global user.signingkey <KEY_ID>
```

Sign all commits by default

```
git config --global commit.gpgsign true
```

Sign a specific commit manually

Signing with SSH Keys (Git 2.34+)

SSH signing is simpler than GPG — uses your existing SSH key:

```
# Configure SSH signing
```

```
git config --global gpg.format ssh
```

```
git config --global user.signingkey ~/.ssh/id_ed25519.pub
```

```
git config --global commit.gpgsign true
```

```
# Create an allowed_signers file (for verification)
```

```
echo "jane@example.com namespaces=\"git\" $(cat ~/.ssh/id_ed25519.pub)" \  
>> ~/.ssh/allowed_signers
```

```
git config --global gpg.ssh.allowedSignersFile ~/.ssh/allowed_signers
```

```
# Verify a commit
```

GitHub — Verified Badge

Status	What it means
Verified (green)	Commit signature verified by GitHub
Unverified	No signature or signature cannot be verified
Vigilant mode	User opted in — all unsigned commits shown as unverified

```
# Add your GPG public key to GitHub:  
# Settings → SSH and GPG keys → New GPG key  
gpg --armor --export <KEY ID>
```

Security Best Practices for Git

Practice	Why
Never commit secrets (keys, passwords, tokens)	Use <code>.gitignore</code> + secret scanning
Use SSH keys, not passwords	Harder to phish
Enable 2FA on GitHub/GitLab	Protect your account
Sign commits	Verify author identity in the commit chain
Use <code>--force-with-lease</code>	Never silently overwrite others' work

Summary — Enterprise Git

Topic	Key takeaway
Code reviews	Small PRs, clear descriptions, automate what you can
Branch protection	Require PR + CI checks before merge
Monorepos	Great for sharing code; use sparse checkout for performance
Git LFS	Use for binaries > 1 MB (images, videos, models)
Supply chain security	GPG or SSH signing — required for supply chain